

EE708: Assignment 5

Q1. Number of Weights in Neural Network

Let the network have:

- N input neurons
- H hidden neurons
- C output neurons

Each connection corresponds to one weight.

(a) Only Input $\rightarrow$ Hidden and Hidden $\rightarrow$ Output

Step 1: Input to Hidden

Each of the N input neurons connects to each of the H hidden neurons:

$$\text{Weights} = N \times H$$

Step 2: Hidden to Output

Each of the H hidden neurons connects to each of the C output neurons:

$$\text{Weights} = H \times C$$

Total weights:

$$\boxed{NH + HC}$$

(b) Including Input $\rightarrow$ Output connections

Now an additional direct connection exists from input to output.

Extra connections:

$$N \times C$$

Total weights:

$$\boxed{NH + HC + NC}$$

Insight: Adding skip connections increases representational flexibility.

—

Q2. Comparison of Network A and Network B

Important Observation: Since all layers use **linear activation functions**, the entire network reduces to a single linear transformation:

$$W_3(W_2(W_1x)) = Wx$$

Thus, both networks can only represent **linear functions**.

(a) Advantage of Network A (Shallow Network)

Key Idea: No unnecessary constraints

- Network A directly learns a transformation: $\hat{y} = Wx$, where $W \in \mathbb{R}^{100 \times 100}$
- It can represent **any linear mapping** (full rank possible)
- Fewer layers $\Rightarrow$ simpler optimization and lower computational overhead

Conclusion:

Network A can represent all linear transformations without restriction and is simpler to train

(b) Advantage of Network B (Deep Network)

Key Idea: Low-rank factorization (parameter efficiency)

Network B computes: $\hat{y} = W_3W_2x$

where: $W_2 \in \mathbb{R}^{10 \times 100}$, $W_3 \in \mathbb{R}^{100 \times 10}$

Thus, $\hat{y} = Wx$ with $W = W_3W_2$

Important consequence: $\text{rank}(W) \leq 10$

- This imposes a **low-rank constraint**
- Acts as an **implicit regularizer**
- Reduces number of parameters: $(100 \times 10 + 10 \times 100) \ll (100 \times 100)$

Conclusion:

Network B is more parameter-efficient and enforces low-rank structure. This improves generalization

Depth does NOT increase expressivity under linear activations

It only changes how the linear mapping is parameterized

Network B cannot represent all linear transformations ($\text{rank} \leq 10$)

Q3. Finding Bias for Logistic Neuron

Given:

- Inputs: $x_1 = -1, x_2 = 1$
- Weights: $w_1 = 0.1, w_2 = 0.5$
- Output: $\phi(v) = 0.73$
- Activation: Logistic with $a = 2$

Step 1: Activation Function

$$\phi(v) = \frac{1}{1 + e^{-2v}}$$

Step 2: Linear Combination

$$v = x_1 w_1 + x_2 w_2 + b$$

$$v = (-1)(0.1) + (1)(0.5) + b = 0.4 + b$$

Step 3: Use Output

$$0.73 = \frac{1}{1 + e^{-2v}}$$

Invert:

$$1 + e^{-2v} = \frac{1}{0.73}$$

$$e^{-2v} = \frac{1}{0.73} - 1 = 0.3699$$

$$-2v = \ln(0.3699)$$

$$v = -\frac{1}{2} \ln(0.3699) \approx 0.4973$$

Step 4: Solve for Bias

$$0.4 + b = 0.4973$$

$$\boxed{b \approx 0.0973}$$

—

Q4. Perceptrons for Logic Gates

Activation Function

We use a perceptron with step activation:

$$y = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases} \quad \text{where } z = w^T x + b$$

(a) NOT Gate

Truth Table:

x	NOT(x)
0	1
1	0

Goal:

$$\begin{cases} z \geq 0 & \text{when } x = 0 \\ z < 0 & \text{when } x = 1 \end{cases}$$

Perceptron equation:

$$z = wx + b$$

Conditions:

$$x = 0 : \quad b \geq 0$$

$$x = 1 : \quad w + b < 0$$

Choose:

$$w = -1, \quad b = 0.5$$

Verification:

$$x = 0 : \quad z = 0.5 \geq 0 \Rightarrow y = 1$$

$$x = 1 : \quad z = -1 + 0.5 = -0.5 < 0 \Rightarrow y = 0$$

Perceptron correctly implements NOT

(b) NAND Gate

Truth Table:

x_1	x_2	NAND
0	0	1
0	1	1
1	0	1
1	1	0

Goal:

$$\begin{cases} z \geq 0 & \text{for } (0,0), (0,1), (1,0) \\ z < 0 & \text{for } (1,1) \end{cases}$$

Perceptron equation:

$$z = w_1x_1 + w_2x_2 + b$$

Conditions:

$$(0, 0) : b \geq 0$$

$$(1, 0) : w_1 + b \geq 0$$

$$(0, 1) : w_2 + b \geq 0$$

$$(1, 1) : w_1 + w_2 + b < 0$$

Choose:

$$w_1 = -1, \quad w_2 = -1, \quad b = 1.5$$

Verification:

$$(0, 0) : z = 1.5 \geq 0 \Rightarrow 1$$

$$(1, 0) : z = 0.5 \geq 0 \Rightarrow 1$$

$$(0, 1) : z = 0.5 \geq 0 \Rightarrow 1$$

$$(1, 1) : z = -0.5 < 0 \Rightarrow 0$$

Geometric Interpretation

The perceptron defines a linear boundary:

$$w_1x_1 + w_2x_2 + b = 0$$

For NAND:

$$x_1 + x_2 = 1.5$$

This line separates (1, 1) from all other points.

Conclusion

NOT and NAND are linearly separable and can be implemented using a single perceptron

Proper choice of weights and bias ensures correct classification

—

Q5. Perceptron for Parity of Three Inputs

Key Insight:

The parity function for three inputs is defined as:

$$y = x_1 \oplus x_2 \oplus x_3$$

It outputs 1 when the number of 1's is odd, and 0 otherwise.

Truth Table:

x_1	x_2	x_3	y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Observation:

The parity function is **not linearly separable**. Therefore,

A single perceptron cannot implement parity

Hence, we construct a **multi-layer perceptron (MLP)**.

Architecture

- Input layer: x_1, x_2, x_3
- Hidden layer: 4 neurons
- Output layer: 1 neuron
- Activation function: Step function

$$\text{step}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Hidden Layer Construction

Each hidden neuron detects one specific input pattern where parity = 1.

Neuron h_1 : detects (0, 0, 1)

$$h_1 = \text{step}(-x_1 - x_2 + x_3 - 0.5)$$

Neuron h_2 : detects (0, 1, 0)

$$h_2 = \text{step}(-x_1 + x_2 - x_3 - 0.5)$$

Neuron h_3 : detects (1, 0, 0)

$$h_3 = \text{step}(x_1 - x_2 - x_3 - 0.5)$$

Neuron h_4 : detects (1, 1, 1)

$$h_4 = \text{step}(x_1 + x_2 + x_3 - 2.5)$$

—

Interpretation

Each hidden neuron acts as a **pattern detector**:

- h_1 activates only for 001
- h_2 activates only for 010
- h_3 activates only for 100
- h_4 activates only for 111

These are exactly the cases where parity = 1.

—

Output Layer

The output neuron performs an OR-like aggregation:

$$y = \text{step}(h_1 + h_2 + h_3 + h_4 - 0.5)$$

Explanation:

- If any hidden neuron = 1 $\Rightarrow y = 1$
- If all are 0 $\Rightarrow y = 0$

—

Final Model Summary

- Hidden layer weights and biases:

$$h_1 : (-1, -1, +1), \quad b = -0.5$$

$$h_2 : (-1, +1, -1), \quad b = -0.5$$

$$h_3 : (+1, -1, -1), \quad b = -0.5$$

$$h_4 : (+1, +1, +1), \quad b = -2.5$$

- Output layer:

$$w = (1, 1, 1, 1), \quad b = -0.5$$

—

Conclusion

Parity requires a multi-layer network with hidden pattern detectors

This construction correctly implements $x_1 \oplus x_2 \oplus x_3$

—

Q6. Weight Update for MLP with ReLU

We consider a single hidden layer MLP with:

- Input: $x \in \mathbb{R}^D$
 - Hidden layer: H units
 - Output: scalar $\hat{y}$
-

Forward Pass

Hidden layer (pre-activation):

$$z_i^{(1)} = \sum_{j=1}^D W_{ij}^{(1)} x_j + b_i^{(1)}$$

Activation (ReLU):

$$h_i = \text{ReLU}(z_i^{(1)}) = \max(0, z_i^{(1)})$$

Output layer:

$$\hat{y} = \sum_{i=1}^H W_i^{(2)} h_i + b^{(2)}$$

Loss function (MSE):

$$L = \frac{1}{2}(y - \hat{y})^2$$

Backward Pass (Derivation)

We apply the chain rule step-by-step.

Step 1: Gradient w.r.t. Output

$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} \left(\frac{1}{2}(y - \hat{y})^2 \right) = -(y - \hat{y})$$

Step 2: Output Layer Gradients

(a) Weights $W_i^{(2)}$:

$$\begin{aligned}\frac{\partial \hat{y}}{\partial W_i^{(2)}} &= h_i \\ \Rightarrow \frac{\partial L}{\partial W_i^{(2)}} &= -(y - \hat{y})h_i\end{aligned}$$

—

(b) Bias $b^{(2)}$:

$$\begin{aligned}\frac{\partial \hat{y}}{\partial b^{(2)}} &= 1 \\ \Rightarrow \frac{\partial L}{\partial b^{(2)}} &= -(y - \hat{y})\end{aligned}$$

—

Step 3: Hidden Layer Gradients

(a) Gradient w.r.t. hidden activation h_i :

$$\begin{aligned}\frac{\partial \hat{y}}{\partial h_i} &= W_i^{(2)} \\ \Rightarrow \frac{\partial L}{\partial h_i} &= -(y - \hat{y})W_i^{(2)}\end{aligned}$$

—

(b) ReLU derivative:

$$\frac{\partial h_i}{\partial z_i^{(1)}} = \begin{cases} 1 & z_i^{(1)} > 0 \\ 0 & z_i^{(1)} \leq 0 \end{cases}$$

—

(c) Gradient w.r.t. pre-activation $z_i^{(1)}$:

$$\frac{\partial L}{\partial z_i^{(1)}} = -(y - \hat{y})W_i^{(2)} \cdot \mathbb{I}(z_i^{(1)} > 0)$$

—

(d) Gradient w.r.t. weights $W_{ij}^{(1)}$:

$$\begin{aligned}\frac{\partial z_i^{(1)}}{\partial W_{ij}^{(1)}} &= x_j \\ \Rightarrow \frac{\partial L}{\partial W_{ij}^{(1)}} &= -(y - \hat{y})W_i^{(2)} \cdot \mathbb{I}(z_i^{(1)} > 0) \cdot x_j\end{aligned}$$

—

(e) Gradient w.r.t. bias $b_i^{(1)}$:

$$\frac{\partial z_i^{(1)}}{\partial b_i^{(1)}} = 1$$
$$\Rightarrow \frac{\partial L}{\partial b_i^{(1)}} = -(y - \hat{y}) W_i^{(2)} \cdot \mathbb{I}(z_i^{(1)} > 0)$$

—

Final Update Equations

Let $\delta = (y - \hat{y})$.

Using gradient descent:

Output layer:

$$W_i^{(2)} \leftarrow W_i^{(2)} + \eta \delta h_i$$

$$b^{(2)} \leftarrow b^{(2)} + \eta \delta$$

Hidden layer:

$$W_{ij}^{(1)} \leftarrow W_{ij}^{(1)} + \eta \delta W_i^{(2)} \mathbb{I}(z_i^{(1)} > 0) x_j$$

$$b_i^{(1)} \leftarrow b_i^{(1)} + \eta \delta W_i^{(2)} \mathbb{I}(z_i^{(1)} > 0)$$

—

Final Insight

Gradient flows backward scaled by weights and gated by ReLU derivative

ReLU acts as a switch: gradients pass only when $z_i^{(1)} > 0$